Yat-CAShed: 基于学习缓存重用性的并行作业多核缓存感知分配算法

1 算法概述

Yat-CAShed是一种创新的缓存感知分配算法，专门设计用于多核系统中有向无环图(DAG)任务的高效调度。该算法的核心思想是通过预测模型来估算作业在不同核心上的执行时间加速比，并基于此进行最优的核心分配决策，从而最大化缓存利用率并减少DAG任务的完成时间。

算法的主要创新在于引入了缓存重用距离(Cache Recency)的概念，通过Cache Recency Profile (CRP)模型来预测作业在特定核心上的执行时间改进情况，实现了系统级别的缓存性能优化。

2 系统模型与基础定义

2.1 硬件架构模型

系统包含m个同构核心$\Lambda = \{\lambda_1, \lambda_2, ..., \lambda_m\}$和$\eta$层包容式缓存层次结构$L = \{L_1, L_2, ..., L_\eta\}$。在包容式缓存中，$L_1$缓存的内容同时存在于$L_2$和$L_3$缓存中。

典型的三层缓存架构中：

- L_1缓存：每个核心独享
- L_2缓存：集群内核心共享
- L_3缓存：所有核心共享

2.2 任务模型

系统中的任务组织为n个周期性DAG任务$\Gamma = \{\tau_1, \tau_2, ..., \tau_n\}$。每个DAG任务$\tau_i$定义为：

$$\tau_i = \{G_i = (v_i, E_i), T_i, P_i, w_i\}$$

其中：

- $G_i = (v_i, E_i)$：DAG的内部结构，v_i为节点集合，E_i为边集合
- T_i：任务周期
- P_i：任务优先级
- w_i：总工作负载，计算公式为$w_i = \sum_{u_{i,j} \in v_i} C_{i,j}$

每个节点$u_{i,j} \in v_i$具有最坏情况执行时间(WCET) $C_{i,j}$。作业(job)表示DAG某次释放中的节点实例，记为$u_{i,j,v}$，其中v表示第v次释放。

2.3 缓存重用距离与预测模型

2.3.1 缓存重用距离定义

缓存重用距离是Yat-CAShed算法的核心概念，定义为：

$$r(u_j, L_x) = \Delta NoUC(u_{j,v}, u_{j,v-1}, L_x)$$

其中：

- L_x：第x级缓存
- $u_{j,v}$：节点u_j的第v次作业实例
- $\Delta NoUC(\cdot)$：两次连续访问之间其他任务访问的唯一缓存行数量

2.3.2 基于时间的重用距离近似

由于新指令数量通常是时间的线性递增函数，缓存重用距离可以通过以下公式近似：

$$r(u_j, L_x) = g(t(u_{j,v}) - t(u_{j,v-1}), L_x)$$

其中$g(\cdot)$函数表示在时间间隔内其他任务执行时间的总和。

2.3.3 Cache Recency Profile (CRP)模型

CRP描述了相对于WCET的加速比与重用距离的关系。该模型具有以下特性：

性质1：CRP产生的执行时间估算随重用距离单调递增

性质2：CRP可以用n条连续线段的分段线性函数建模

CRP模型通过测量不同重用距离值下的实际执行时间来学习获得，具有以下三个主要趋势：

- 核心重用(Core Recency)：L_1缓存命中情况
- 集群重用(Cluster Recency)：L_2缓存命中情况
- 系统重用(System Recency)：L_3缓存命中情况

2.3.4 核心分配机制

对于作业u_j和候选核心λ_k，加速比$S(u_j, \lambda_k, H, CRP)$的计算步骤如下：

1. 检查L_1缓存命中条件：

- 在λ_k上找到前一个实例$u_{j,v-1} \in H(\lambda_k)$
- 重用距离$r(u_{j,v}, L_1)$小于核心重用阈值

2. 如果满足L_1缓存命中，加速比计算为：

$$S(u_j, \lambda_k, H, CRP) = (1 - CRP(r(u_j, L_1))) \times C_j$$

3. 如果L_1缓存未命中，依次检查L_2和L_3缓存，使用相同方法计算。

Yat-CAShed采用两个核心分配规则：

最大加速优先(Maximum Speedup First, MSF)：MSF构建加速表(Speedup Table, SUT)，包含前δ个作业在每个空闲核心上的加速比估算。算法总是将作业u_j分配给具有最高$S(\cdot)$值的可用核心λ_k。

最小缓存影响优先(Least Cache Impact First, LCIF)：当作业在多个核心上具有相同加速比时，LCIF计算分配对其他作业缓存收益的影响：

$$Imp(u_j, \lambda_k) = \sum_{\forall u_x \in H(\lambda_k)} (S(u_x, \lambda_k, H, CRP) - S(u_x, \lambda_k, H', CRP))$$

其中H'表示将作业u_j添加到核心λ_k后的分配历史表。

作业调度遵循以下优先级： 1. 按DAG优先级排序(全局固定优先级调度) 2. 相同DAG优先级内，按WCET降序排列 3. 在调度点最多分派δ个就绪作业到δ个空闲核心

2.3.5 算法实现流程

Yat-CAShed算法在每个调度点执行以下步骤：

1. **作业识别**：根据调度顺序和空闲核心数量确定待分派作业集合Q_{sched}
2. **加速表构建**：计算每个待分派作业在每个空闲核心上的加速比$S(\cdot)$
3. **最优分配决策**：

- 应用MSF规则选择具有最高加速比的作业-核心对
- 如果存在多个相同加速比的候选核心，应用LCIF规则选择影响最小的核心

Algorithm 1 Yat-CAShed在线分配算法

Require: Q_{ready}, Λ^*, H, CRP

$Q_{sched} = sort(Q_{ready}).first(||\Lambda^*||)$
$S = init_SUT()$
for $u_j \in Q_{sched}$ **do**
  for $\lambda_k \in \Lambda^*$ **do**
    $S(u_j, \lambda_k) = S(u_j, \lambda_k, H, CRP)$
  end for
end for
while $Q_{sched} \neq \emptyset$ **do**
  $(u_j, \Lambda^{\neg}) = \arg\max\{S(u_j, \lambda_k) | \forall u_j \in Q_{sched}\}$
  if $|\Lambda^{\neg}| == 1$ **then**
    $\lambda_k = \Lambda^{\neg}(1)$
  else
    $\lambda_k = \arg\min\{Imp(u_j, \lambda_k) | k \in \Lambda^{\neg}\}$
  end if
  $\alpha_j = \lambda_k$
  $Q_{sched}.remove(u_j)$
  $S.remove(u_j, \lambda_k)$
  $H.add(u_j, \lambda_k)$
end while

4. **状态更新**：更新加速表S、历史表H和调度队列Q_{sched}

5. **迭代执行**：重复上述过程直到所有待分派作业都被分配

算法的时间复杂度为$O(m^2 + m \times (m \times n))$，其中$m$为系统核心数量，$n$为单个核心上检查的最大作业数量。算法维护分配历史表H来跟踪已分配作业的指定核心，并通过预定义的预测模型而非复杂的在线计算来保证效率。

通过这种机制，Yat-CAShed算法能够在运行时动态优化作业到核心的分配，最大化缓存利用率，从而显著减少DAG任务的完成时间并提高系统整体吞吐量。